

OneKhusa Integration Manual

Hosted Checkout, Webhooks,

Onekhusa Team

May 2026

Contents

1	System Architecture	2
1.1	Project Directory Structure	2
1.2	Merchant Configuration (appsettings.json)	2
2	Hosted Checkout Implementation	2
2.1	Checkout Initiation (TicketsController.cs)	2
2.2	Closing the Loop (WebhooksController.cs)	3
3	Tiered Disbursement Logic	3
3.1	2-Eye: Raw Byte Batch Upload	3
4	Conclusion	3

1 System Architecture

The OneTicket API acts as a secure bridge between the React frontend and the OneKhusa financial network.

1.1 Project Directory Structure

```
1 OneTicket.API/  
2 |-- Controllers/  
3 |   |-- DisbursementsController.cs    (Payout Logic)  
4 |   |-- TicketsController.cs          (Checkout Initiation)  
5 |   +-- WebhooksController.cs         (Payment Sync)  
6 |-- DTOs/  
7 |   |-- SinglePayoutRequest.cs  
8 |   |-- OneKhusaWebhook.cs            (Status Mapping)  
9 |   +-- TransferBatchRequest.cs  
10 |-- Services/  
11 |   +-- TicketTracker.cs              (Memory State)  
12 |-- Program.cs                        (Startup Config)  
13 +-- appsettings.json                  (Merchant Keys)
```

Listing 1: Standard ASCII Directory Layout

1.2 Merchant Configuration (appsettings.json)

```
1 {  
2   "OneKhusa": {  
3     "OrganizationId": "FYHONTVWODXK",  
4     "MerchantAccountNumber": 79619974,  
5     "ApiKey": "sandbox_FWkYLGiwaOMyG909mUeEWuf2GupfeM..",  
6     "ApiSecret": "-c2YBsNLOBFVHOzI7Ik6OI9n0tcAARi3374sg7WGnf02Jlv6IJ..",  
7     "WebhookSecret": "NTEK1KbI7J9tfYwAlyj-G3cP79MImodBY3f5DfGCuq..."  
8   }  
9 }
```

2 Hosted Checkout Implementation

The Collection flow utilizes the Hosted Checkout API to generate a secure redirection link.

2.1 Checkout Initiation (TicketsController.cs)

This implementation manually constructs the HTTP request to obtain the `paymentTransactionId` (ptid) for the redirect.

```
1 [HttpPost("buy/{eventId}")]  
2 public async Task<IActionResult> BuyTicket(string eventId, [FromQuery] string email) {  
3     var payload = new {  
4         authentication = new { apiKey = _apiKey, apiSecret = _apiSecret },  
5         merchant = new { organisationId = _orgId, merchantAccountNumber = 79619974 },  
6         payment = new { sourceReferenceNumber = "OT" + DateTime.Now.Ticks, amount = 2500.00, capturedBy = email  
7     },  
8     route = new { redirectionUrl = "http://localhost:5173/", callbackApiUrl = "https://your-ngrok.dev/wc-api/onekhusa_webhook" }  
9 };  
10  
11 var request = new HttpRequestMessage(HttpMethod.Post, "https://api.onekhusa.com/sandbox/v1/checkout/rtp/initiate");  
12 request.Headers.Add("X-Idempotency-Key", "INIT-" + Guid.NewGuid());  
13 request.Content = JsonContent.Create(payload);  
14  
15 var response = await _client.SendAsync(request);  
16 if (response.IsSuccessStatusCode) {  
17     var result = await response.Content.ReadFromJsonAsync<System.Text.Json.JsonElement>();  
18     string ptid = result.GetProperty("paymentTransactionId").GetString();  
19     return Ok(new { checkoutUrl = $"https://checkout.onekhusa.com/requestToPay/initiate?ptid={ptid}&rn={sourceRef}" });  
20 }
```

```

19     }
20     return BadRequest();
21 }

```

Listing 2: Initiation and ptid Construction

2.2 Closing the Loop (WebhooksController.cs)

Asynchronous payment confirmation is handled via webhooks. Upon receiving an S100 code, the system must notify the Hosted Page to finalize the session.

```

1 [HttpPost("onekhusa_webhook")]
2 public async Task<IActionResult> HandleWebhook([FromBody] OneKhusaWebhook payload) {
3     if (payload.ResponseCode == "S100") {
4         _tracker.UpdateStatus(payload.Metadata.ReferenceNumber, "Paid");
5
6         var notify = new {
7             paymentTransactionId = payload.PaymentTransactionId,
8             status = "SUCCESS",
9             responseCode = "S100"
10        };
11        // Notifies OneKhusa to turn the screen green and redirect the user
12        await _client.PostAsJsonAsync("https://checkout.onekhusa.com/requestToPay/webhook", notify);
13    }
14    return Ok();
15 }

```

Listing 3: Webhook loop-closing implementation

3 Tiered Disbursement Logic

Disbursements are implemented using the ****2-Eye****, ****4-Eye****, and ****6-Eye**** security principles.

3.1 2-Eye: Raw Byte Batch Upload

To maintain binary integrity and avoid "Exact column casing" errors, the system transmits files as raw byte[] arrays.

```

1 [HttpPost("upload-batch")]
2 public async Task<IActionResult> Upload([FromForm] IFormFile file) {
3     using var ms = new MemoryStream();
4     await file.CopyToAsync(ms);
5     var req = new UploadBatchFileRequest {
6         MerchantAccountNumber = 79619974,
7         FileContent = ms.ToArray(),
8         ContentType = Path.GetExtension(file.FileName).ToLower() == ".csv" ? "CSV" : "XLSX"
9     };
10    var res = await _oneKhusa.Transactions.BatchDisbursements.UploadFileAsync(req, "UP-" + Guid.NewGuid());
11    return res.IsSuccess ? Ok(res.Data) : BadRequest(res.Error);
12 }

```

Listing 4: DisbursementsController Upload

4 Conclusion

This reference provides a complete roadmap for OneKhusa integration. By combining raw byte file handling with the Hosted Checkout "Three-Way Handshake," the OneTicket platform ensures reliable, high-performance financial processing.